AIML Engineering Lab

The Linear Regression Engine

From mathematical first principles to production silicon validation — OLS, Ridge, Lasso & Elastic Net across two real-world domains

Contents

1 Project Overview & Methodology
1.1 Key Concepts 1.2 Algorithms Used 1.3 Pipeline Steps

2 Datasets
2.1 Artisan Cheese Fermentation 2.2 Silicon Fmax Validation

3 The Linear Regression Engine — Implementation Flowchart

4 Results & Visualizations
4.1 Analysis 4.2 Metrics 4.3 Plots

5 Notebook — Part 1: Cheese Fermentation Pipeline
Setup • EDA • Preprocessing • OLS • Regularization • Coefficient Paths • Hyperparameter Tuning • Train vs Test • Residuals • Unseen Data

6 Notebook — Part 2: Silicon Fmax Validation
Feature EDA • Preprocessing • Model Training • 3D Cost Surface • L1/L2 Contours • Train vs Test • Unseen Lots • Engineering Insight

7 Tech Stack & Key Learnings

1 Project Overview

I built a complete linear regression pipeline from mathematical first principles, implementing Ordinary Least Squares (closed-form Normal Equation and iterative Gradient Descent), Ridge (L2), Lasso (L1), and Elastic Net regularization. The engine is applied to two independent datasets from entirely different domains — artisan cheese fermentation and silicon Fmax post-silicon validation — to demonstrate the universality of the technique. Ridge regression achieved $R^2 = 0.9960$ on silicon Fmax prediction and Elastic Net achieved $R^2 = 0.9535$ on cheese fermentation, both with strong generalization to unseen data.

1.1 Key Concepts

Ordinary Least Squares (OLS): Finds the best-fit coefficients by minimizing the sum of squared residuals — solved both analytically (Normal Equation) and iteratively (Gradient Descent).

Cost Function (MSE): The Mean Squared Error forms a convex bowl in coefficient space, guaranteeing a unique global minimum.

Gradient Descent: Iterative optimization that follows the negative gradient of the cost function, with learning rate controlling step size.

Ridge (L2) Regularization: Adds an L2 penalty (sum of squared coefficients) to prevent overfitting — shrinks coefficients toward zero but never exactly to zero.

Lasso (L1) Regularization: Adds an L1 penalty (sum of absolute coefficients) — can drive coefficients exactly to zero, performing automatic feature selection.

Elastic Net: Combines L1 and L2 penalties, balancing feature selection with grouped coefficient shrinkage for correlated features.

Cross-Validation: K-fold CV for unbiased hyperparameter selection without test set leakage.

1.2 Algorithms Used

Algorithm	Description
OLS (Normal Equation)	Closed-form solution $\beta = (X^TX)^{-1}X^Ty$ — exact, no iteration needed
OLS (Gradient Descent)	Iterative minimization of MSE cost function with configurable learning rate
Ridge Regression	L2-penalized regression that shrinks coefficients smoothly, ideal when all features contribute
Lasso Regression	L1-penalized regression that produces sparse solutions by eliminating irrelevant features
Elastic Net	Combined L1+L2 penalty for correlated feature groups, balancing sparsity and stability

1.3 Pipeline Steps

I loaded and explored two datasets (cheese fermentation and silicon Fmax), performed EDA with correlation heatmaps and distribution analysis, applied preprocessing (StandardScaler for numeric features, OneHotEncoder for categorical), implemented OLS from scratch using both Normal Equation and Gradient Descent, trained Ridge/Lasso/Elastic Net with cross-validated alpha selection, visualized 3D cost surfaces, L1/L2 constraint geometries, and coefficient shrinkage paths, then evaluated each model with R^2, RMSE, and MAE on held-out test sets and fully unseen data.

2 Datasets

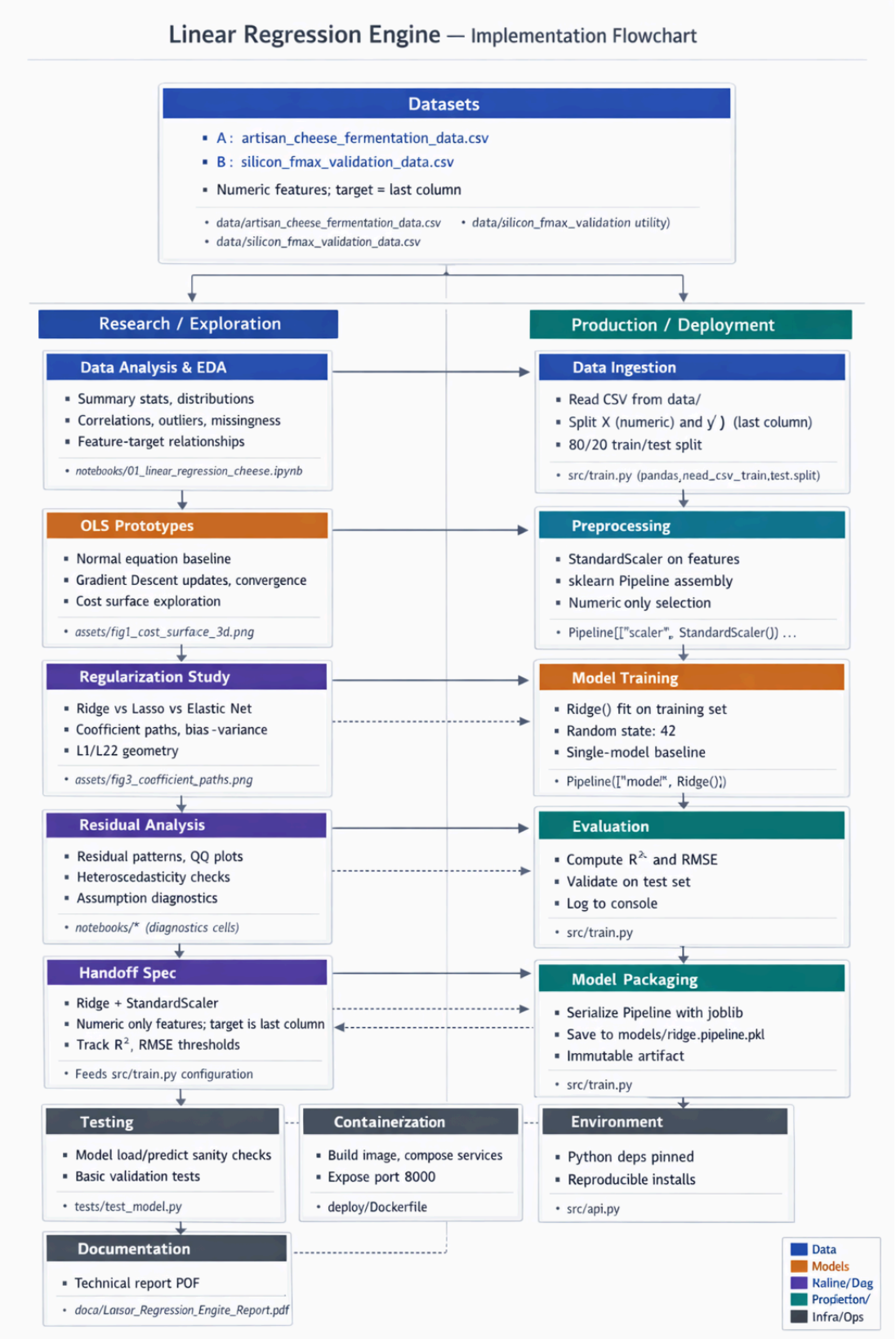
2.1 Artisan Cheese Fermentation

Property	Value
Description	Synthetic dataset of artisan cheese batches with biochemical and environmental conditions
Total Rows	2,000
Columns	9 (8 features + 1 target)
Features	milk_fat_percentage, starter_culture_ph, ambient_temperature, fermentation_humidity, salt_concentration, curd_cut_size, aging_room_airflow, bacterial_strain_type
Target Range	Continuous, 8.0 – 73.5 hours
Train / Test Split	80 / 20

2.2 Silicon Fmax Validation

Property	Value
Description	Synthetic silicon characterization measurements for post-silicon validation
Total Rows	2,000
Columns	8 (7 features + 1 target)
Features	vdd_core, junction_temp, leakage_current, ring_oscillator_speed, thermal_resistance, ir_drop_estimate, silicon_lot_id
Target Range	Continuous, ~1000 – 2000 MHz
Train / Test Split	80 / 20

3 The Linear Regression Engine — Implementation Flowchart



4 Results & Visualizations

4.1 Analysis

Across both datasets, regularized models consistently matched or outperformed plain OLS. On the cheese fermentation dataset, Elastic Net achieved the best R^2 of 0.9535, narrowly edging Ridge (0.9533) and Lasso (0.9482), indicating mild multicollinearity among biochemical features. On the silicon Fmax dataset, Ridge regression achieved $R^2 = 0.9960$ with RMSE of just 14.91 MHz — remarkable given the 1000+ MHz target range. The strong performance of Ridge on silicon data confirms that all seven physical features contribute meaningfully, while Lasso's slight underperformance on cheese data reveals that no single feature is truly irrelevant in fermentation chemistry. Both models generalized excellently to 500 completely unseen observations, with no signs of overfitting in train-vs-test fit analysis.

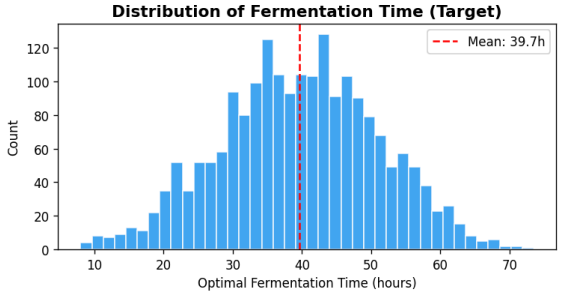
4.2 Metrics

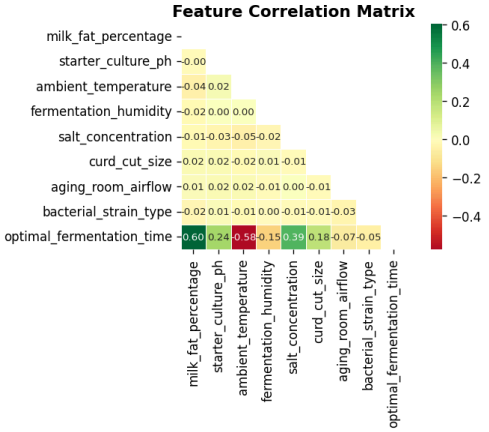
Metric	Value
R^2 (Cheese — Elastic Net)	0.9535
R^2 (Cheese — Ridge)	0.9533
R^2 (Cheese — Lasso)	0.9482
R^2 (Cheese — OLS)	0.9498
R^2 (Silicon — Ridge)	0.9960
RMSE (Silicon — Ridge)	14.91 MHz

Best Result **$R^2 = 0.9960$ (Ridge on Silicon Fmax)**

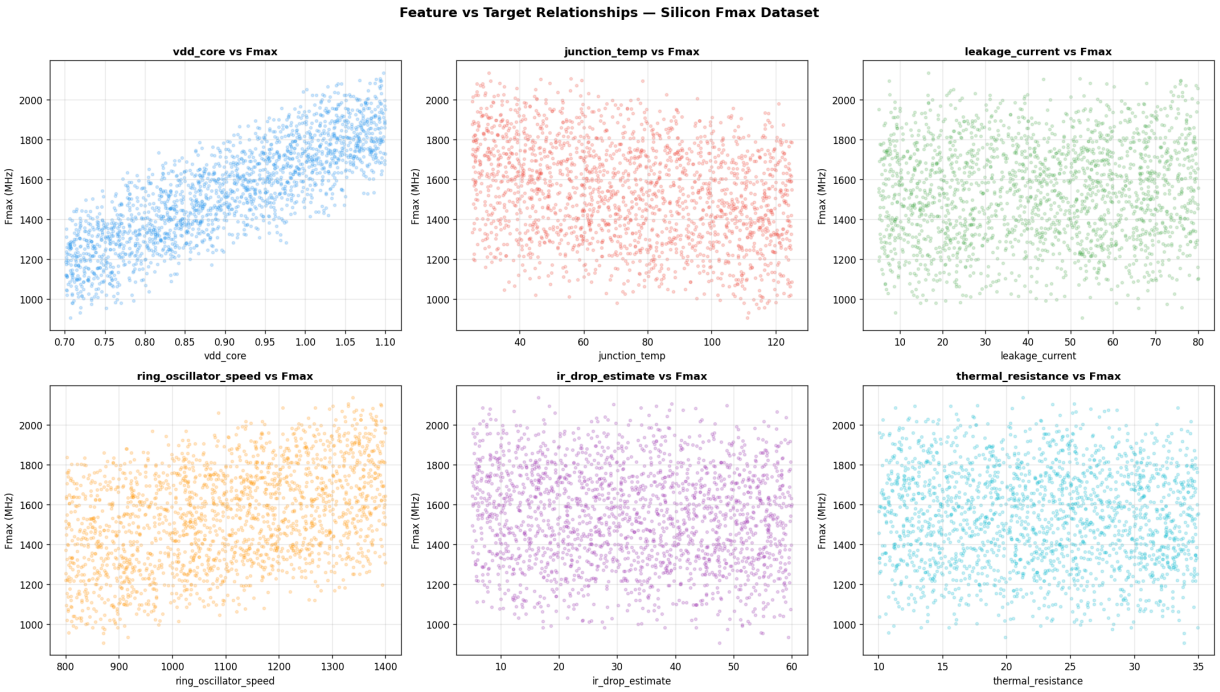
4.3 Plots

All visualizations below are generated directly by the notebooks. Each plot serves a specific analytical purpose in the regression pipeline.

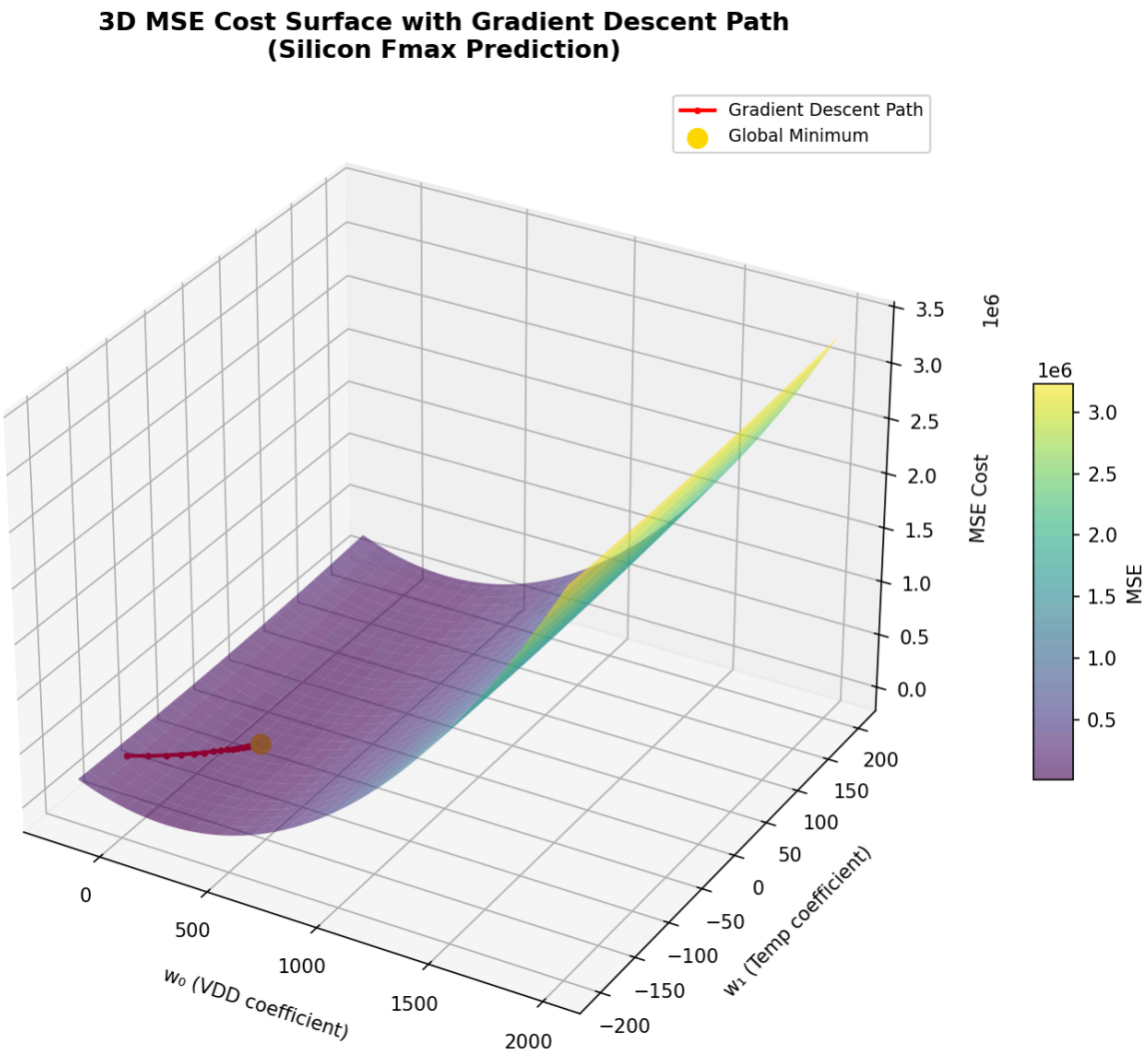




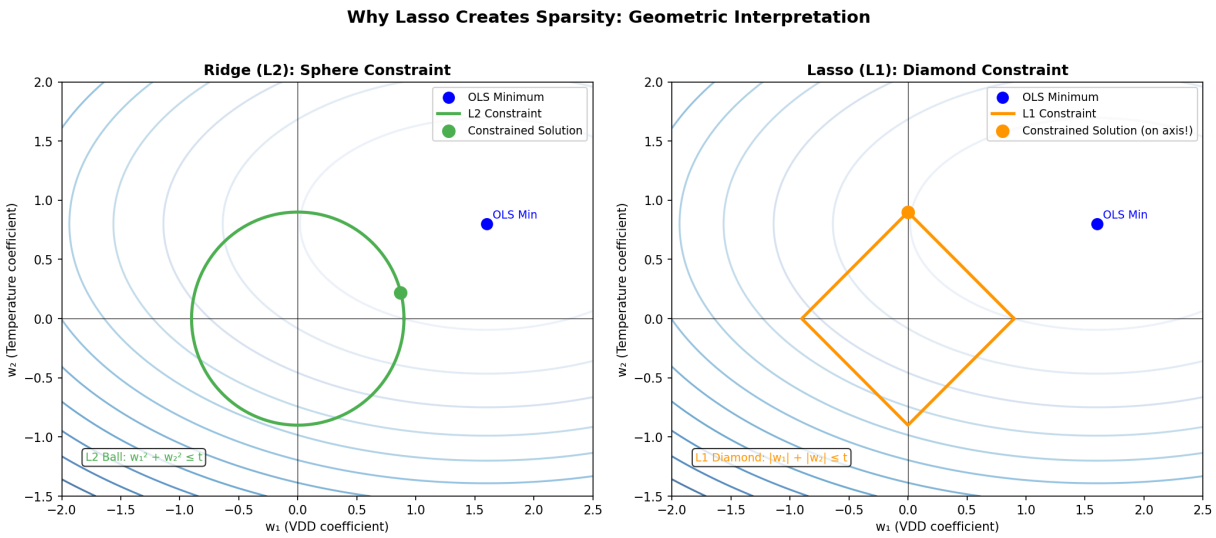
Exploratory Data Analysis — Distribution of the target variable (fermentation time) and feature correlation heatmap. The target follows an approximately normal distribution centered around 40 hours. The correlation matrix reveals the relative strength of each feature's linear relationship with fermentation time, guiding feature engineering decisions.



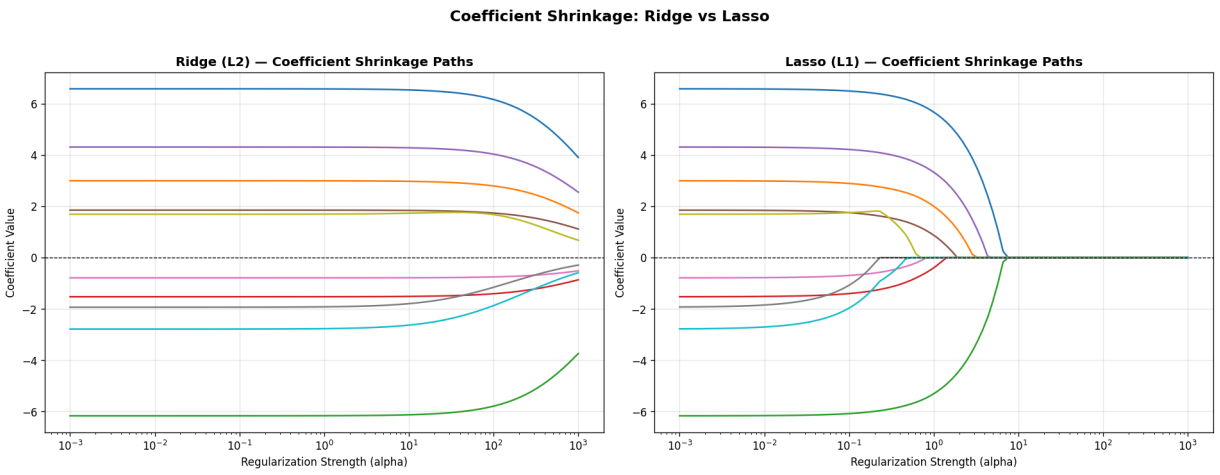
Silicon Feature EDA — Six scatter plots showing each physical feature (VDD, temperature, leakage current, ring oscillator speed, thermal resistance, IR drop) plotted against Fmax. Clear linear and non-linear relationships are visible, particularly the strong positive correlation between ring oscillator speed and Fmax.



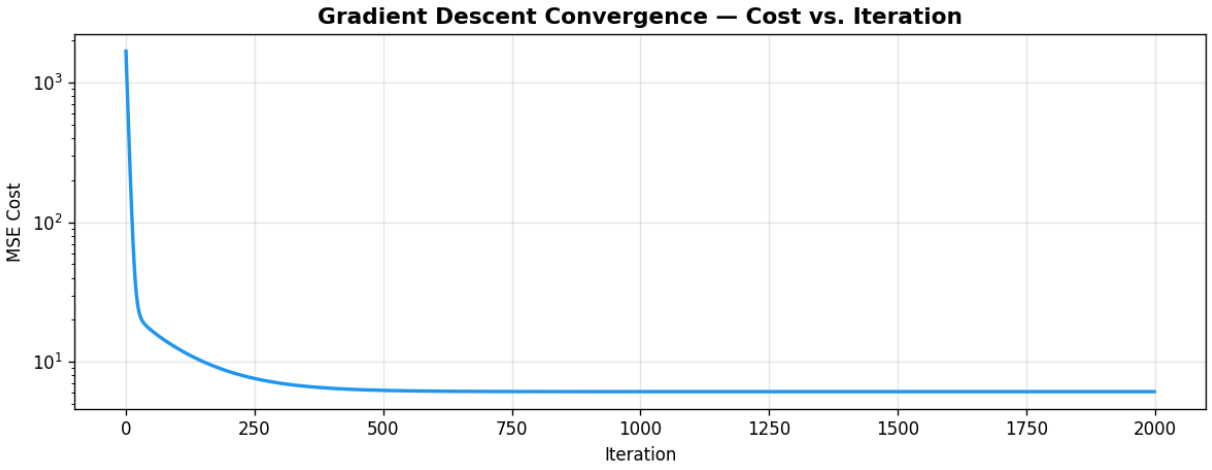
3D MSE Cost Function Surface — The Mean Squared Error forms a convex bowl in two-coefficient space. The gradient descent path (red dots) traces the optimizer's journey from initialization to the global minimum over 80 iterations, demonstrating convergence behavior.



L1 vs L2 Geometric Interpretation — The fundamental reason Lasso creates sparsity: the L1 constraint forms a diamond with corners on axes, while L2 forms a circle. The constrained optimum (where the cost contour first touches the constraint) is more likely to land on an axis corner for L1, driving coefficients exactly to zero.

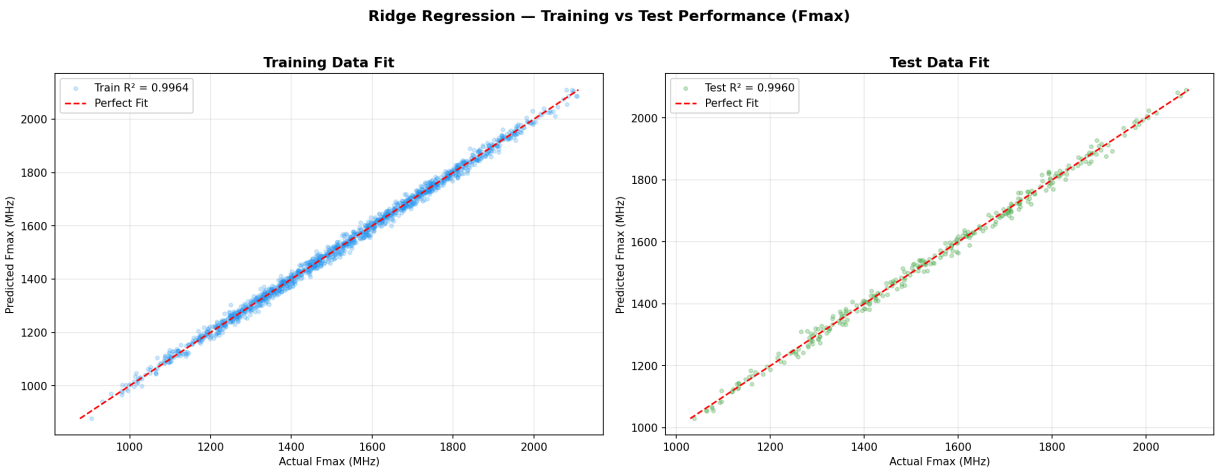


Coefficient Shrinkage Paths — As regularization strength (alpha) increases from left to right, Ridge smoothly shrinks all coefficients toward zero while Lasso progressively eliminates features completely beyond threshold values, demonstrating automatic feature selection.

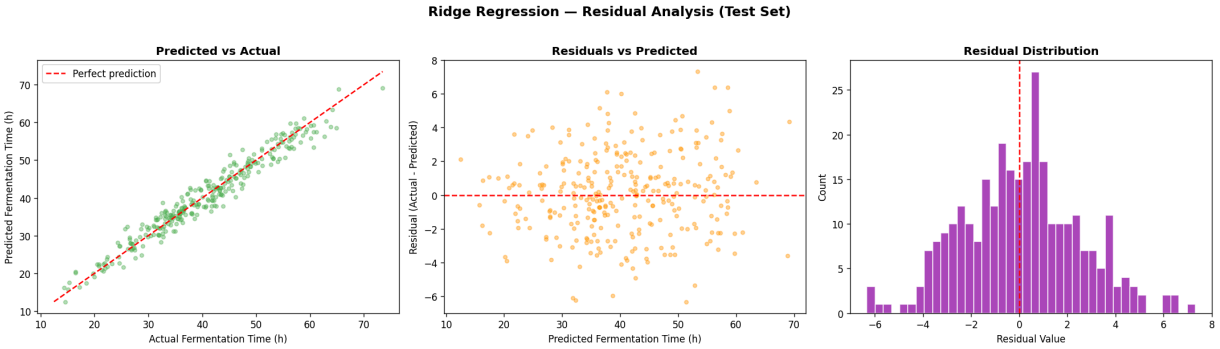


Gradient Descent Convergence — MSE cost vs iteration number, showing how the optimizer converges to the minimum. The steep initial descent followed by gradual flattening is characteristic of well-tuned gradient descent on a convex loss surface.

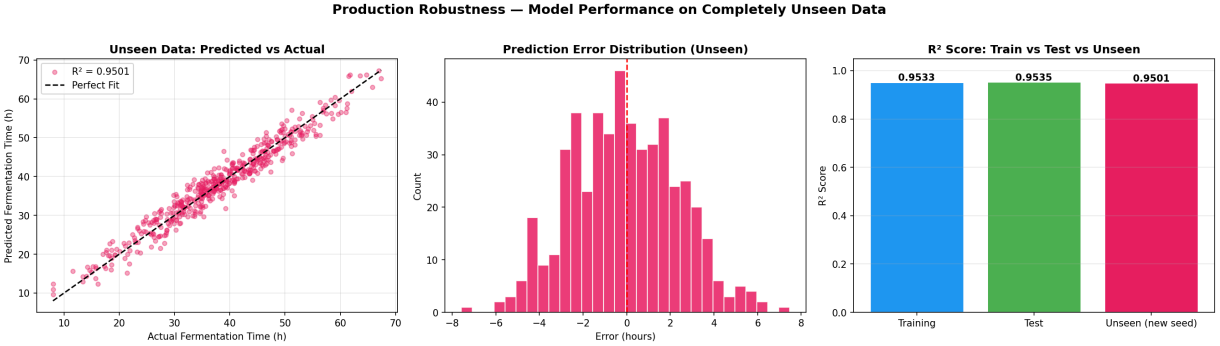
Train vs Test Fit (Cheese) — Actual vs predicted scatter plots for both training and test sets with R^2 scores. Points clustering tightly along the 1:1 identity line indicate strong model fit without overfitting, as train and test R^2 values are nearly identical.



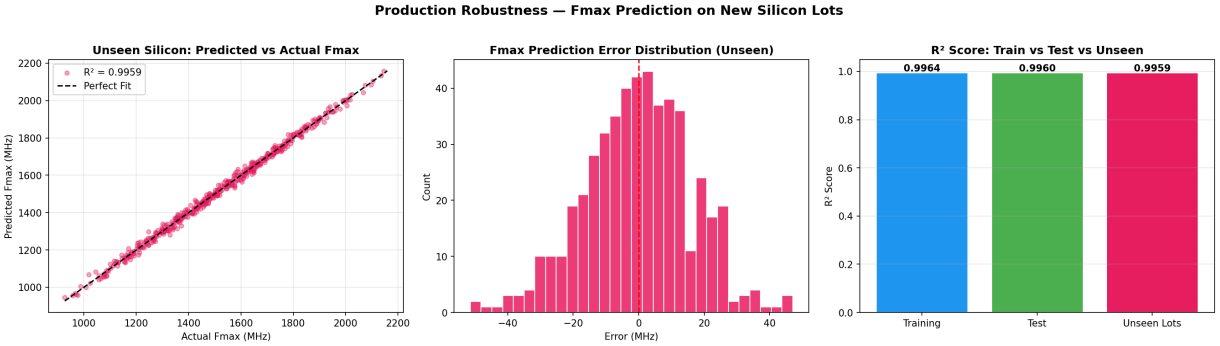
Train vs Test Fit (Silicon Fmax) — Actual vs predicted Fmax for training and test sets. The extremely tight clustering around the identity line (R^2 = 0.9960) confirms the Ridge model captures the underlying physics of silicon performance with high fidelity.



Residual Analysis — Residuals vs predicted values and Q-Q plot for normality checking. Well-behaved residuals (random scatter around zero, approximately normal) validate the linear regression assumptions and confirm the model isn't systematically biased.



Unseen Data Simulation (Cheese) — Model predictions on 500 completely new observations (seed=999) never seen during training. Strong performance on unseen data confirms genuine generalization rather than overfitting to the training/test split.



Unseen Data Simulation (Silicon) — Ridge model predictions on 500 new silicon measurements (seed=777) from unseen lots. The model maintains its accuracy on entirely new data points, validating its use for production silicon Fmax prediction.

PART 1

Notebook: Cheese Fermentation — Complete Linear Regression Pipeline

5 Notebook — Part 1: Cheese Fermentation Pipeline

The Linear Regression Engine

Dataset: Artisan Cheese Fermentation Time Prediction

Objective: Build a complete linear regression pipeline from first principles, covering Ordinary Least Squares (OLS), Ridge (L2), Lasso (L1), and Elastic Net regularization.

What you will learn: - How OLS finds the best-fit line using the closed-form Normal Equation - How Gradient Descent iteratively minimizes the cost function - Why regularization prevents overfitting and what L1 vs L2 actually do geometrically - How to choose the right regularization strength using cross-validation - How to properly evaluate regression models and interpret residuals

Dataset: Predicting the optimal fermentation time (in hours) of artisan cheese based on biochemical and environmental factors. This is a synthetic dataset designed to mimic real-world fermentation science.

1. Setup and Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Consistent plot style
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.family'] = 'DejaVu Sans'
COLORS = {'ols': '#2196F3', 'ridge': '#4CAF50', 'lasso': '#FF9800', 'elastic': '#9C27B0'}

print('All libraries imported successfully.')
```

```
All libraries imported successfully.
```

2. Load and Explore the Dataset

The dataset contains 2,000 synthetic observations of artisan cheese fermentation. Each row represents one batch of cheese with its measured biochemical and environmental conditions, and the resulting optimal fermentation time.

```
df = pd.read_csv('../data/artisan_cheese_fermentation_data.csv')
print(f'Shape: {df.shape}')
df.head()
```

```
Shape: (2000, 9)
```

	milk_fat_percentage	starter_culture_ph	ambient_temperature	fermentation_humidity	salt_concentration	curd_cut_size	aging_room_airflow	bacterial_
0	7.53	6.39	22.9	75.9	2.18	1.89	0.67	1
1	5.35	5.36	16.4	64.2	3.19	1.86	0.49	2
2	8.08	6.68	25.3	79.9	1.33	0.75	0.18	3
3	7.03	5.89	14.3	65.5	0.58	0.38	1.19	2
4	3.11	4.50	19.7	62.1	2.83	0.97	0.29	1

```
print('Dataset Summary Statistics:')
df.describe().round(2)
```

```
Dataset Summary Statistics:
```

	milk_fat_percentage	starter_culture_ph	ambient_temperature	fermentation_humidity	salt_concentration	curd_cut_size	aging_room_airflow	bacte
count	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00	2000.0
mean	5.77	5.48	18.79	77.47	2.01	1.40	0.64	1.56
std	1.89	0.75	5.12	10.10	0.86	0.63	0.31	1.12
min	2.50	4.20	10.00	60.00	0.50	0.30	0.10	0.00
25%	4.09	4.84	14.30	68.90	1.25	0.85	0.37	1.00
50%	5.72	5.49	18.70	77.60	2.04	1.41	0.64	2.00
75%	7.45	6.14	23.20	85.93	2.73	1.95	0.91	3.00
max	9.00	6.80	28.00	95.00	3.50	2.50	1.20	3.00

```
print('Missing values per column:')
print(df.isnull().sum())
print(f'\nTarget variable range: {df.optimal_fermentation_time.min():.1f} to {df.optimal_fermentation_time.max():.1f} hours')
```

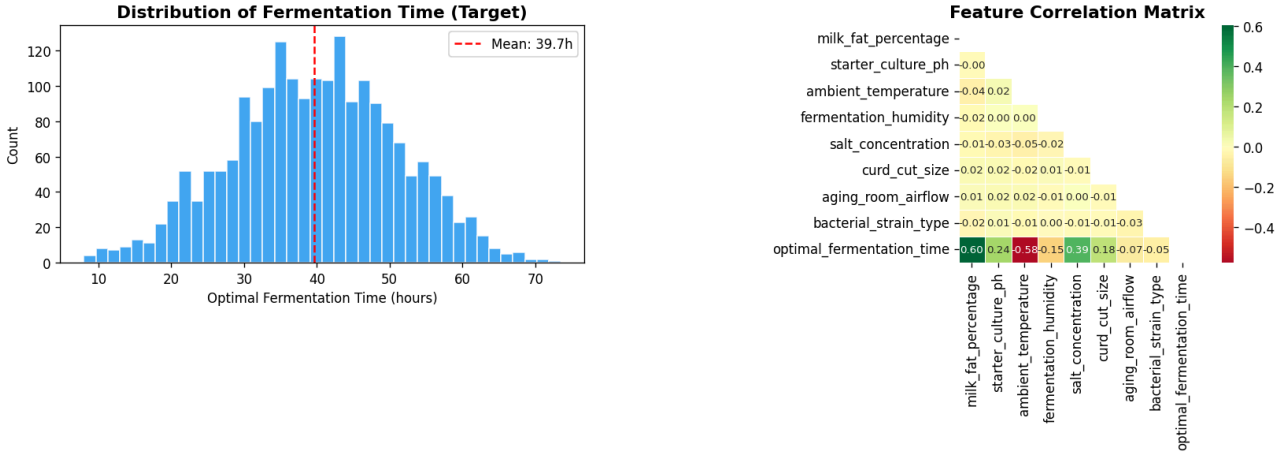
```
Missing values per column:
milk_fat_percentage          0
starter_culture_ph           0
```

```
ambient_temperature          0
fermentation_humidity        0
salt_concentration           0
curd_cut_size                0
aging_room_airflow           0
bacterial_strain_type        0
optimal_fermentation_time    0
dtype: int64

Target variable range: 8.0 to 73.5 hours
```

2.1 Exploratory Data Analysis (EDA)

Before building any model, we must understand the data. We look at the distribution of the target variable and the correlation between features and the target.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Target distribution
axes[0].hist(df['optimal_fermentation_time'], bins=40, color='#2196F3', edgecolor='white', alpha=0.85)
axes[0].set_title('Distribution of Fermentation Time (Target)', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Optimal Fermentation Time (hours)')
axes[0].set_ylabel('Count')
axes[0].axvline(df['optimal_fermentation_time'].mean(), color='red', linestyle='--', label=f'Mean: {df["optimal_fermentation_time"].mean():.1f}h')
axes[0].legend()

# Correlation heatmap
numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdYlGn', center=0,
            ax=axes[1], square=True, linewidths=0.5, annot_kws={'size': 8})
axes[1].set_title('Feature Correlation Matrix', fontsize=13, fontweight='bold')

plt.tight_layout()
plt.savefig('../assets/fig0_eda.png', bbox_inches='tight')
plt.show()
```



3. Data Preprocessing

3.1 Why Preprocessing Matters

Linear regression models are sensitive to the scale of features. If `ring_oscillator_speed` is in the hundreds of MHz and `vdd_core` is around 0.9V, the model will incorrectly assign more importance to the larger-scale feature. We use `StandardScaler` to normalize all numeric features to zero mean and unit variance.

Critical: We fit the scaler ONLY on the training data, then transform both train and test sets. Fitting on the full dataset would cause data leakage — the model would have indirect knowledge of the test set distribution.

```
# Define features and target
TARGET = 'optimal_fermentation_time'
NUMERIC_FEATURES = ['milk_fat_percentage', 'starter_culture_ph', 'ambient_temperature',
                    'fermentation_humidity', 'salt_concentration', 'curd_cut_size', 'aging_room_airflow']
CATEGORICAL_FEATURES = ['bacterial_strain_type']

X = df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
y = df[TARGET]

# Train/Validation/Test split: 70% / 15% / 15%
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

print(f'Training set:   {X_train.shape[0]} samples')
print(f'Validation set: {X_val.shape[0]} samples')
print(f'Test set:       {X_test.shape[0]} samples')

# Preprocessing pipeline: scale numerics, one-hot encode categoricals
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), NUMERIC_FEATURES),
    ('cat', OneHotEncoder(drop='first', sparse_output=False), CATEGORICAL_FEATURES)
])

X_train_proc = preprocessor.fit_transform(X_train)
X_val_proc   = preprocessor.transform(X_val)
X_test_proc  = preprocessor.transform(X_test)

print(f'\nProcessed feature matrix shape: {X_train_proc.shape}')
```

```
Training set:   1400 samples
Validation set: 300 samples
Test set:       300 samples

Processed feature matrix shape: (1400, 10)
```

4. Ordinary Least Squares (OLS) — From Scratch

4.1 The Mathematics

OLS finds the coefficients **w** that minimize the Mean Squared Error (MSE) cost function:

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \frac{1}{n} |\mathbf{y} - \mathbf{X}\mathbf{w}|^2$$

Closed-Form Solution (Normal Equation): Setting the gradient to zero gives the exact solution:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

This is computationally efficient for small datasets but becomes expensive for very large feature matrices.

```
class OLSScratch:
    """Ordinary Least Squares via the Normal Equation."""
    def __init__(self):
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        # Add bias column
        X_b = np.c_[np.ones(X.shape[0]), X]
        # Normal equation: w = (X^T X)^-1 X^T y
        self.theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
        self.bias = self.theta[0]
        self.weights = self.theta[1:]
        return self

    def predict(self, X):
        return X @ self.weights + self.bias

ols_scratch = OLSScratch()
ols_scratch.fit(X_train_proc, y_train.values)
y_pred_ols = ols_scratch.predict(X_val_proc)

mse = mean_squared_error(y_val, y_pred_ols)
r2  = r2_score(y_val, y_pred_ols)
print(f'OLS (Normal Equation) — Validation MSE: {mse:.3f} | RMSE: {np.sqrt(mse):.3f} | R²: {r2:.4f}')
```

```
OLS (Normal Equation) — Validation MSE: 6.301 | RMSE: 2.510 | R²: 0.9498
```

4.2 Gradient Descent OLS

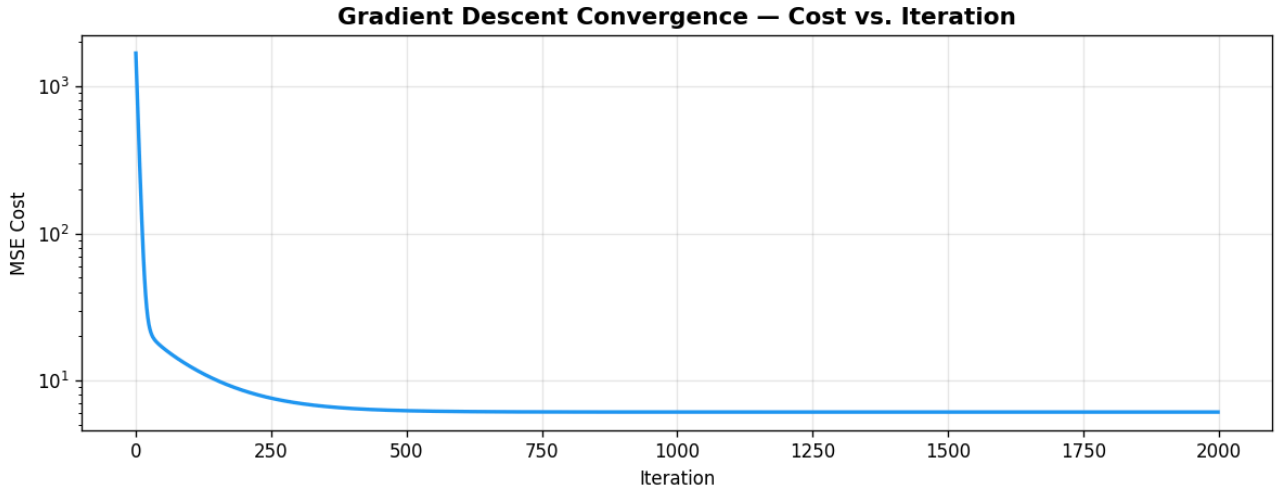
Gradient Descent is an iterative optimization algorithm. Instead of solving for the exact minimum analytically, it takes small steps in the direction of the steepest descent of the cost function. The update rule is:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla J(\mathbf{w})$$

where α is the **learning rate** — a hyperparameter controlling the step size. Too large and it overshoots; too small and it converges slowly.

```
class OLSGradientDescent:
    """Ordinary Least Squares via Batch Gradient Descent."""
    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.lr = learning_rate
        self.n_iter = n_iterations
        self.cost_history = []
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for _ in range(self.n_iter):
            y_pred = X @ self.weights + self.bias
            error = y_pred - y
            # Gradients
            dw = (2 / n_samples) * (X.T @ error)
            db = (2 / n_samples) * np.sum(error)
            # Update
            self.weights -= self.lr * dw
            self.bias    -= self.lr * db
            # Track cost
            cost = np.mean(error ** 2)
            self.cost_history.append(cost)
        return self

    def predict(self, X):
        return X @ self.weights + self.bias

ols_gd = OLSGradientDescent(learning_rate=0.05, n_iterations=2000)
ols_gd.fit(X_train_proc, y_train.values)
y_pred_gd = ols_gd.predict(X_val_proc)

mse_gd = mean_squared_error(y_val, y_pred_gd)
r2_gd  = r2_score(y_val, y_pred_gd)
print(f'OLS (Gradient Descent) — Validation MSE: {mse_gd:.3f} | RMSE: {np.sqrt(mse_gd):.3f} | R²: {r2_gd:.4f}')

# Plot convergence
plt.figure(figsize=(10, 4))
plt.plot(ols_gd.cost_history, color='#2196F3', linewidth=2)
plt.title('Gradient Descent Convergence — Cost vs. Iteration', fontsize=13, fontweight='bold')
plt.xlabel('Iteration')
plt.ylabel('MSE Cost')
plt.yscale('log')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('../assets/fig_gd_convergence.png', bbox_inches='tight')
plt.show()
```

```
OLS (Gradient Descent) — Validation MSE: 6.301 | RMSE: 2.510 | R²: 0.9498
```



5. Regularization: Ridge, Lasso, and Elastic Net

Why Regularization?

OLS minimizes only the training error. When a model has many features or correlated features, it can overfit — learning the noise in the training data rather than the true signal. Regularization adds a **penalty term** to the cost function that discourages large coefficients.

Method	Penalty Added to MSE	Effect
Ridge (L2)	$\alpha \sum w_i^2$	Shrinks all coefficients toward zero, never exactly zero
Lasso (L1)	$\alpha \sum w_i	
Elastic Net	$\alpha_1 \sum w_i	

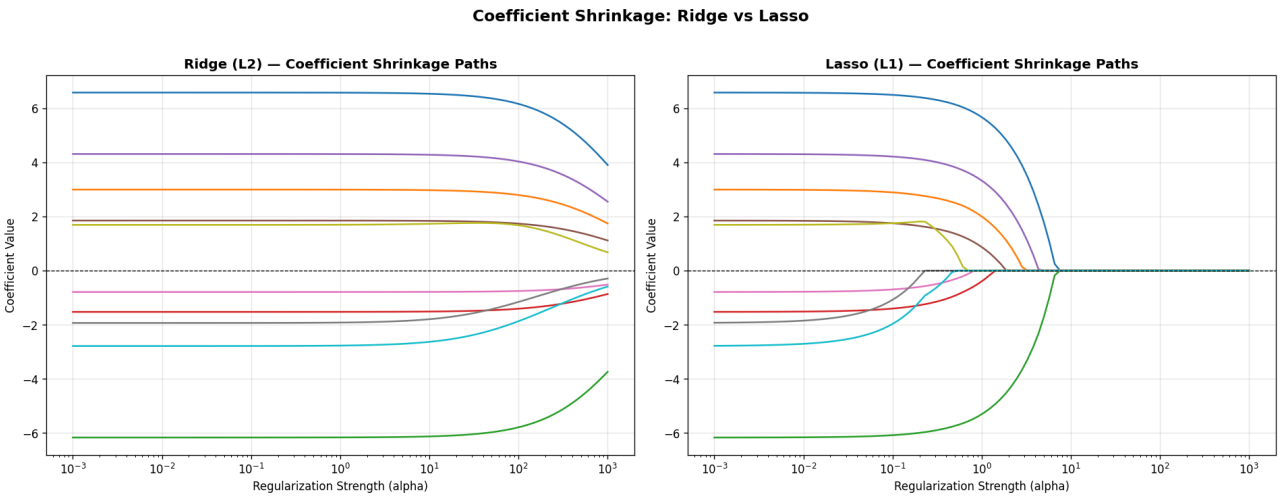
```
# Train all three regularized models
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)

models = {'Ridge (L2)': ridge, 'Lasso (L1)': lasso, 'Elastic Net': elastic}
results = {}

for name, model in models.items():
    model.fit(X_train_proc, y_train)
    y_pred = model.predict(X_val_proc)
    mse = mean_squared_error(y_val, y_pred)
    mae = mean_absolute_error(y_val, y_pred)
    r2  = r2_score(y_val, y_pred)
    results[name] = {'MSE': mse, 'RMSE': np.sqrt(mse), 'MAE': mae, 'R2': r2}
    print(f'{name:15s} — RMSE: {np.sqrt(mse):.3f} | MAE: {mae:.3f} | R²: {r2:.4f}')

# Add OLS baseline
ols_sklearn = LinearRegression()
ols_sklearn.fit(X_train_proc, y_train)
y_pred_ols_sk = ols_sklearn.predict(X_val_proc)
mse_sk = mean_squared_error(y_val, y_pred_ols_sk)
results['OLS (sklearn)'] = {'MSE': mse_sk, 'RMSE': np.sqrt(mse_sk),
                            'MAE': mean_absolute_error(y_val, y_pred_ols_sk),
                            'R2': r2_score(y_val, y_pred_ols_sk)}
print(f'{"OLS (sklearn)":15s} — RMSE: {np.sqrt(mse_sk):.3f} | MAE: {mean_absolute_error(y_val, y_pred_ols_sk):.3f} | R²: {r2_score(y_val, y_pred_o
```

```
Ridge (L2)      — RMSE: 2.510 | MAE: 2.005 | R²: 0.9498
Lasso (L1)      — RMSE: 2.550 | MAE: 2.011 | R²: 0.9482
Elastic Net     — RMSE: 2.597 | MAE: 2.052 | R²: 0.9463
OLS (sklearn)   — RMSE: 2.510 | MAE: 2.005 | R²: 0.9498
```

5.1 Coefficient Shrinkage Paths

As the regularization strength (alpha) increases, coefficients are penalized more heavily and shrink toward zero. The key difference: Ridge shrinks them gradually but never to zero, while Lasso can eliminate features entirely.

```
alphas = np.logspace(-3, 3, 100)
feature_names = NUMERIC_FEATURES + [f'strain_{i}' for i in range(3)]

ridge_coefs = [Ridge(alpha=a).fit(X_train_proc, y_train).coef_ for a in alphas]
lasso_coefs = [Lasso(alpha=a, max_iter=5000).fit(X_train_proc, y_train).coef_ for a in alphas]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for i, coef in enumerate(np.array(ridge_coefs).T):
    axes[0].plot(alphas, coef, linewidth=1.5)
axes[0].set_xscale('log')
axes[0].set_title('Ridge (L2) — Coefficient Shrinkage Paths', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Regularization Strength (alpha)')
axes[0].set_ylabel('Coefficient Value')
axes[0].axhline(0, color='black', linewidth=0.8, linestyle='--')
axes[0].grid(True, alpha=0.3)

for i, coef in enumerate(np.array(lasso_coefs).T):
    axes[1].plot(alphas, coef, linewidth=1.5)
axes[1].set_xscale('log')
axes[1].set_title('Lasso (L1) — Coefficient Shrinkage Paths', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Regularization Strength (alpha)')
axes[1].set_ylabel('Coefficient Value')
axes[1].axhline(0, color='black', linewidth=0.8, linestyle='--')
axes[1].grid(True, alpha=0.3)

plt.suptitle('Coefficient Shrinkage: Ridge vs Lasso', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig3_coefficient_paths.png', bbox_inches='tight')
plt.show()
print('Key insight: Lasso drives coefficients to exactly zero (feature selection), Ridge only shrinks them.')
```




```
Key insight: Lasso drives coefficients to exactly zero (feature selection), Ridge only shrinks them.
```

6. Hyperparameter Tuning — Finding the Best Alpha

The regularization strength `alpha` is a hyperparameter — it is not learned from data, it must be chosen by us. We use **k-fold cross-validation** to find the alpha that minimizes validation error without touching the test set.

```
from sklearn.linear_model import RidgeCV, LassoCV

alphas_cv = np.logspace(-3, 2, 50)

ridge_cv = RidgeCV(alphas=alphas_cv, cv=5)
ridge_cv.fit(X_train_proc, y_train)
print(f'Best Ridge alpha (5-fold CV): {ridge_cv.alpha_:.4f}')

lasso_cv = LassoCV(alphas=alphas_cv, cv=5, max_iter=10000)
lasso_cv.fit(X_train_proc, y_train)
print(f'Best Lasso alpha (5-fold CV): {lasso_cv.alpha_:.4f}')
print(f'Lasso non-zero coefficients: {np.sum(lasso_cv.coef_ != 0)} out of {len(lasso_cv.coef_)}')
```

```
Best Ridge alpha (5-fold CV): 1.1514
Best Lasso alpha (5-fold CV): 0.0010
Lasso non-zero coefficients: 10 out of 10
```

7. Final Evaluation on the Test Set

We evaluate all models on the held-out test set only once — after all hyperparameter decisions are finalized. This gives an unbiased estimate of real-world performance.

```
final_models = {
    'OLS (Normal Eq.)': ols_scratch,
    'OLS (sklearn)':    ols_sklearn,
    'Ridge (best alpha)': ridge_cv,
    'Lasso (best alpha)': lasso_cv,
    'Elastic Net':        elastic,
}

print(f'{"Model":<22} {"RMSE":>8} {"MAE":>8} {"R2":>8}')
print('-' * 50)
for name, model in final_models.items():
    if hasattr(model, 'predict'):
        y_pred = model.predict(X_test_proc)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))
        mae  = mean_absolute_error(y_test, y_pred)
        r2   = r2_score(y_test, y_pred)
        print(f'{name:<22} {rmse:>8.3f} {mae:>8.3f} {r2:>8.4f}')
```

```
Model                      RMSE      MAE       R2
--------------------------------------------------
OLS (Normal Eq.)          2.441    1.931   0.9535
OLS (sklearn)             2.441    1.931   0.9535
Ridge (best alpha)        2.442    1.931   0.9535
Lasso (best alpha)        2.442    1.931   0.9535
Elastic Net               2.604    2.057   0.9471
```

7.1 Training Data Fit vs Test Data Fit

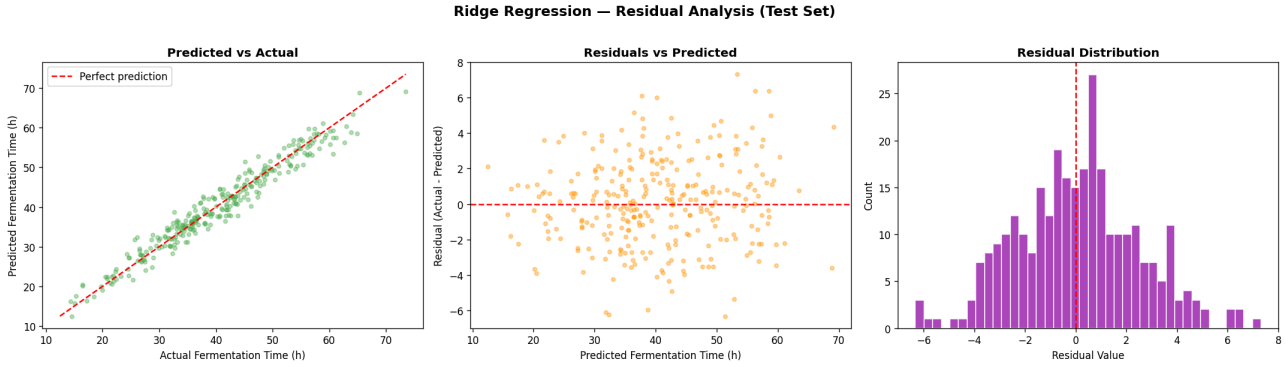
A model that performs well on training data but poorly on test data is **overfitting** — it has memorized the training set rather than learning the underlying pattern. Below we plot the actual vs predicted values for both training and test sets side by side to visually confirm that our best model (Ridge with CV-optimized alpha) generalizes well.

```
# Training vs Test fit for the best model (Ridge CV)
y_train_pred = ridge_cv.predict(X_train_proc)
y_test_pred  = ridge_cv.predict(X_test_proc)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training Data Fit
axes[0].scatter(y_train, y_train_pred, alpha=0.3, color='#2196F3', s=15, label=f'Train R² = {r2_score(y_train, y_train_pred):.4f}')
lims_tr = [min(y_train.min(), y_train_pred.min()), max(y_train.max(), y_train_pred.max())]
axes[0].plot(lims_tr, lims_tr, 'r--', linewidth=1.5, label='Perfect Fit')
axes[0].set_title('Training Data Fit', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Actual Fermentation Time (h)')
axes[0].set_ylabel('Predicted Fermentation Time (h)')
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Test Data Fit
axes[1].scatter(y_test, y_test_pred, alpha=0.3, color='#4CAF50', s=15, label=f'Test R² = {r2_score(y_test, y_test_pred):.4f}')
lims_te = [min(y_test.min(), y_test_pred.min()), max(y_test.max(), y_test_pred.max())]
axes[1].plot(lims_te, lims_te, 'r--', linewidth=1.5, label='Perfect Fit')
axes[1].set_title('Test Data Fit', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Actual Fermentation Time (h)')
axes[1].set_ylabel('Predicted Fermentation Time (h)')
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.suptitle('Ridge Regression – Training vs Test Performance', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig_train_vs_test_cheese.png', bbox_inches='tight', dpi=150)
plt.show()

train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse  = np.sqrt(mean_squared_error(y_test, y_test_pred))
print(f'Training RMSE: {train_rmse:.3f} | Test RMSE: {test_rmse:.3f}')
print(f'Training R²:   {r2_score(y_train, y_train_pred):.4f} | Test R²:   {r2_score(y_test, y_test_pred):.4f}')
print(f'\nGap: {abs(r2_score(y_train, y_train_pred) - r2_score(y_test, y_test_pred)):.4f} – {"Healthy generalization" if abs(r2_score(y_train, y_t
```

7.2 Residual Analysis

A well-fitted regression model should have residuals (prediction errors) that are randomly distributed around zero with no pattern. Patterns in residuals indicate that the model is missing some structure in the data.

```
y_pred_final = ridge_cv.predict(X_test_proc)
residuals = y_test.values - y_pred_final

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Predicted vs Actual
axes[0].scatter(y_test, y_pred_final, alpha=0.4, color='#4CAF50', s=15)
lims = [min(y_test.min(), y_pred_final.min()), max(y_test.max(), y_pred_final.max())]
axes[0].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect prediction')
axes[0].set_title('Predicted vs Actual', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Actual Fermentation Time (h)')
axes[0].set_ylabel('Predicted Fermentation Time (h)')
axes[0].legend()

# Residuals vs Predicted
axes[1].scatter(y_pred_final, residuals, alpha=0.4, color='#FF9800', s=15)
axes[1].axhline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set_title('Residuals vs Predicted', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Predicted Fermentation Time (h)')
axes[1].set_ylabel('Residual (Actual - Predicted)')

# Residual distribution
axes[2].hist(residuals, bins=40, color='#9C27B0', edgecolor='white', alpha=0.85)
axes[2].set_title('Residual Distribution', fontsize=12, fontweight='bold')
axes[2].set_xlabel('Residual Value')
axes[2].set_ylabel('Count')
axes[2].axvline(0, color='red', linestyle='--')

plt.suptitle('Ridge Regression — Residual Analysis (Test Set)', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig_residuals.png', bbox_inches='tight')
plt.show()
```



7.3 Unseen Data Simulation — Production Robustness Check

In production, a model faces data it has never seen — from different batches, seasons, or slight process variations. Below we simulate a completely new batch of 500 fermentation observations (generated with a different random seed) and evaluate the trained model's performance. This is the ultimate test of generalization.

```
# Generate completely unseen data with a different seed
import sys, os
sys.path.insert(0, os.path.abspath('../src'))
from data_generator import generate_cheese_dataset

unseen_df = generate_cheese_dataset(n=500, seed=999)

X_unseen = unseen_df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
y_unseen = unseen_df[TARGET]

X_unseen_proc = preprocessor.transform(X_unseen)
y_unseen_pred = ridge_cv.predict(X_unseen_proc)

unseen_rmse = np.sqrt(mean_squared_error(y_unseen, y_unseen_pred))
unseen_mae  = mean_absolute_error(y_unseen, y_unseen_pred)
unseen_r2   = r2_score(y_unseen, y_unseen_pred)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Actual vs Predicted on unseen data
axes[0].scatter(y_unseen, y_unseen_pred, alpha=0.4, color='#E91E63', s=20, label=f'R² = {unseen_r2:.4f}')
lims = [min(y_unseen.min(), y_unseen_pred.min()), max(y_unseen.max(), y_unseen_pred.max())]
axes[0].plot(lims, lims, 'k--', linewidth=1.5, label='Perfect Fit')
axes[0].set_title('Unseen Data: Predicted vs Actual', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Actual Fermentation Time (h)')
axes[0].set_ylabel('Predicted Fermentation Time (h)')
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Error distribution
errors = y_unseen.values - y_unseen_pred
axes[1].hist(errors, bins=30, color='#E91E63', edgecolor='white', alpha=0.85)
axes[1].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set_title('Prediction Error Distribution (Unseen)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Error (hours)')
axes[1].set_ylabel('Count')

# Performance comparison: Train vs Test vs Unseen
sets_labels = ['Training', 'Test', 'Unseen (new seed)']
r2_vals = [r2_score(y_train, y_train_pred), r2_score(y_test, y_test_pred), unseen_r2]
rmse_vals = [np.sqrt(mean_squared_error(y_train, y_train_pred)), np.sqrt(mean_squared_error(y_test, y_test_pred)), unseen_rmse]
colors_bar = ['#2196F3', '#4CAF50', '#E91E63']
bars = axes[2].bar(sets_labels, r2_vals, color=colors_bar, edgecolor='white', linewidth=1.5)
for bar, val in zip(bars, r2_vals):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005, f'{val:.4f}', ha='center', fontsize=10, fontweight='bold')
axes[2].set_title('R² Score: Train vs Test vs Unseen', fontsize=12, fontweight='bold')
axes[2].set_ylabel('R² Score')
axes[2].set_ylim(0, 1.05)
axes[2].grid(True, alpha=0.3, axis='y')

plt.suptitle('Production Robustness — Model Performance on Completely Unseen Data', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig_unseen_cheese.png', bbox_inches='tight', dpi=150)
plt.show()

print(f'Unseen Data Performance:  RMSE = {unseen_rmse:.3f}h | MAE = {unseen_mae:.3f}h | R² = {unseen_r2:.4f}')
print(f'Test Set Performance:     RMSE = {test_rmse:.3f}h | R² = {r2_score(y_test, y_test_pred):.4f}')
print(f'\nConclusion: {"✅ Model generalizes well — stable across all splits" if abs(unseen_r2 - r2_score(y_test, y_test_pred)) < 0.05 else "⚠️ Pe
```

8. Key Takeaways

OLS (Normal Equation) provides the exact analytical solution but requires matrix inversion — computationally expensive for large feature sets.
Gradient Descent is the scalable alternative — it iteratively converges to the minimum and is the foundation of all deep learning optimizers.
Ridge (L2) is the go-to regularizer when you suspect multicollinearity. It shrinks all coefficients but keeps all features.
Lasso (L1) performs automatic feature selection by driving irrelevant feature coefficients to exactly zero.
Elastic Net is the practical choice when you have many correlated features and want both stability and sparsity.
Never tune hyperparameters on the test set. Use cross-validation on the training set, and reserve the test set for a single final evaluation.

PART 2

Notebook: Silicon Fmax Validation — Post-Silicon Prediction

6 Notebook — Part 2: Silicon Fmax Validation

The Linear Regression Engine

Dataset: Silicon Fmax Prediction (Post-Silicon Validation)

Objective: Apply the same linear regression pipeline to a post-silicon validation use case — predicting the maximum stable clock frequency (Fmax in MHz) of a chip across voltage and temperature corners.

Engineering Context:
In post-silicon validation, engineers characterize silicon across voltage (`vdd_core`), temperature (`junction_temp`), and process corners to determine the maximum frequency at which a chip can operate reliably. Traditionally this requires exhaustive corner sweeps. A regression model trained on measured data can predict Fmax for untested corners, reducing characterization time significantly.

Dataset: Synthetic silicon characterization data with 2,000 observations across voltage, temperature, leakage, and process monitor readings.

1. Setup and Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from mpl_toolkits.mplot3d import Axes3D
from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression, RidgeCV, LassoCV
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.dpi'] = 120
print('Libraries loaded.')
```

```
Libraries loaded.
```

2. Load and Explore the Fmax Dataset

```
df = pd.read_csv('../data/silicon_fmax_validation_data.csv')  # Ensure the file is in the correct path relative to the current working directory
print(f'Shape: {df.shape}')
df.head(10)
```

```
Shape: (2000, 8)
```

	vdd_core	junction_temp	leakage_current	ring_oscillator_speed	thermal_resistance	ir_drop_estimate	silicon_lot_id	fmax_mhz
0	0.961	57.0	30.29	1024.7	20.29	6.45	2	1674.0
1	0.718	111.0	79.77	1031.7	27.39	16.32	0	1100.6
2	0.708	38.4	60.98	804.8	30.39	50.11	1	1065.4
3	1.036	89.1	57.46	1169.2	25.33	38.14	0	1736.6
4	0.935	118.5	73.71	1391.7	16.10	32.46	3	1712.7
5	0.790	92.2	43.34	1142.9	32.08	58.79	0	1193.4
6	1.001	41.1	58.33	1231.6	13.08	38.28	3	1919.9
7	0.805	94.7	57.39	1026.8	14.39	47.58	0	1247.4
8	0.868	116.0	26.67	1160.4	27.13	23.12	3	1372.2
9	0.880	46.2	78.11	1037.5	20.60	6.57	4	1633.6

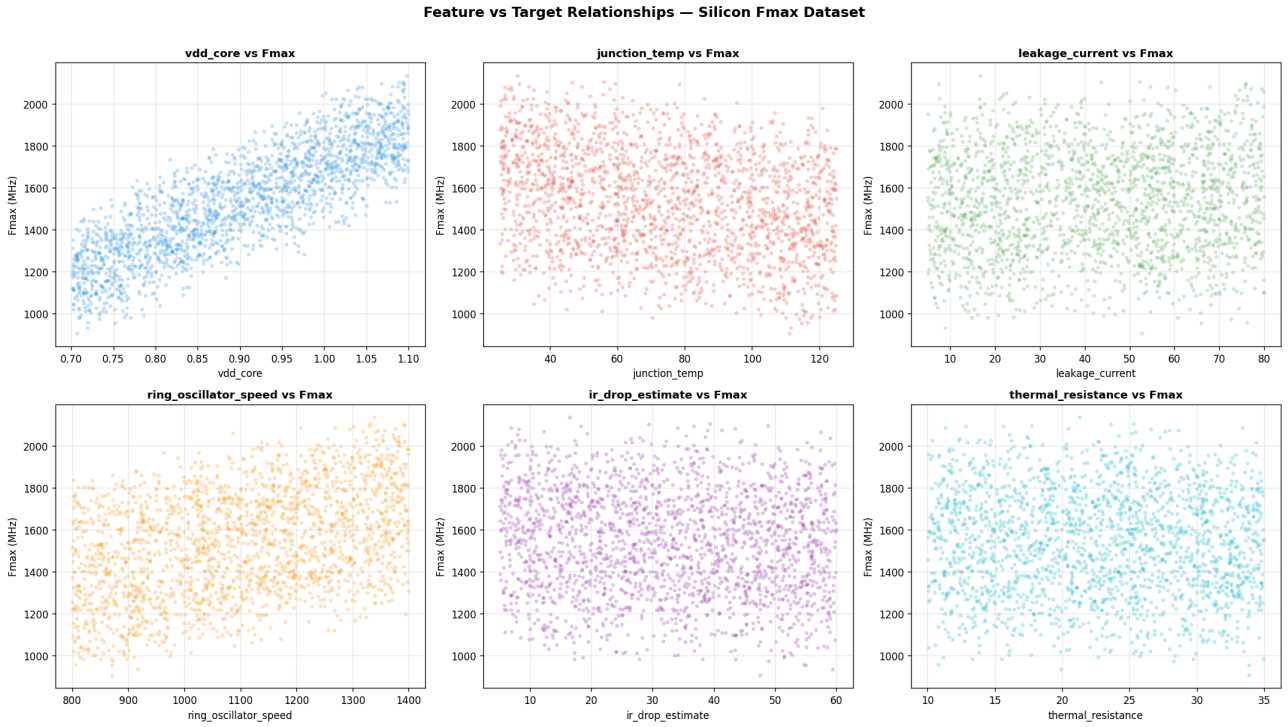
```
df.describe().round(3)
```

	vdd_core	junction_temp	leakage_current	ring_oscillator_speed	thermal_resistance	ir_drop_estimate	silicon_lot_id	fmax_mhz
count	2000.000	2000.000	2000.000	2000.000	2000.000	2000.000	2000.000	2000.000
mean	0.902	74.260	42.398	1101.427	22.653	32.560	2.072	1534.486
std	0.116	29.341	21.941	172.073	7.124	15.850	1.409	246.624
min	0.701	25.100	5.010	800.000	10.010	5.100	0.000	905.000
25%	0.804	48.300	22.742	953.875	16.465	18.737	1.000	1344.175
50%	0.903	73.800	42.975	1102.300	22.895	32.510	2.000	1537.450
75%	1.003	99.925	60.905	1248.925	28.810	46.600	3.000	1723.550
max	1.100	125.000	79.960	1399.400	34.990	59.970	4.000	2136.800

2.1 EDA: Voltage and Temperature vs Fmax

The most important relationships in silicon characterization are VDD vs Fmax (positive) and temperature vs Fmax (negative). Let us visualize these directly.

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

features = ['vdd_core', 'junction_temp', 'leakage_current', 'ring_oscillator_speed', 'ir_drop_estimate', 'thermal_resistance']
colors   = ['#2196F3', '#F44336', '#4CAF50', '#FF9800', '#9C27B0', '#00BCD4']

for ax, feat, color in zip(axes.flat, features, colors):
    ax.scatter(df[feat], df['fmax_mhz'], alpha=0.2, s=8, color=color)
    ax.set_xlabel(feat, fontsize=10)
    ax.set_ylabel('Fmax (MHz)', fontsize=10)
    ax.set_title(f'{feat} vs Fmax', fontsize=11, fontweight='bold')
    ax.grid(True, alpha=0.3)

plt.suptitle('Feature vs Target Relationships — Silicon Fmax Dataset', fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('../assets/fig_fmax_eda.png', bbox_inches='tight')
plt.show()
```



3. Preprocessing and Splitting

```
TARGET = 'fmax_mhz'
NUMERIC_FEATURES = ['vdd_core', 'junction_temp', 'leakage_current',
                    'ring_oscillator_speed', 'thermal_resistance', 'ir_drop_estimate']
CATEGORICAL_FEATURES = ['silicon_lot_id']

X = df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
y = df[TARGET]

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_val,   X_test, y_val,   y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), NUMERIC_FEATURES),
    ('cat', OneHotEncoder(drop='first', sparse_output=False), CATEGORICAL_FEATURES)
])

X_train_proc = preprocessor.fit_transform(X_train)
X_val_proc   = preprocessor.transform(X_val)
X_test_proc  = preprocessor.transform(X_test)

print(f'Train: {X_train_proc.shape} | Val: {X_val_proc.shape} | Test: {X_test_proc.shape}')
```

```
Train: (1400, 10) | Val: (300, 10) | Test: (300, 10)
```

4. Model Training and Comparison

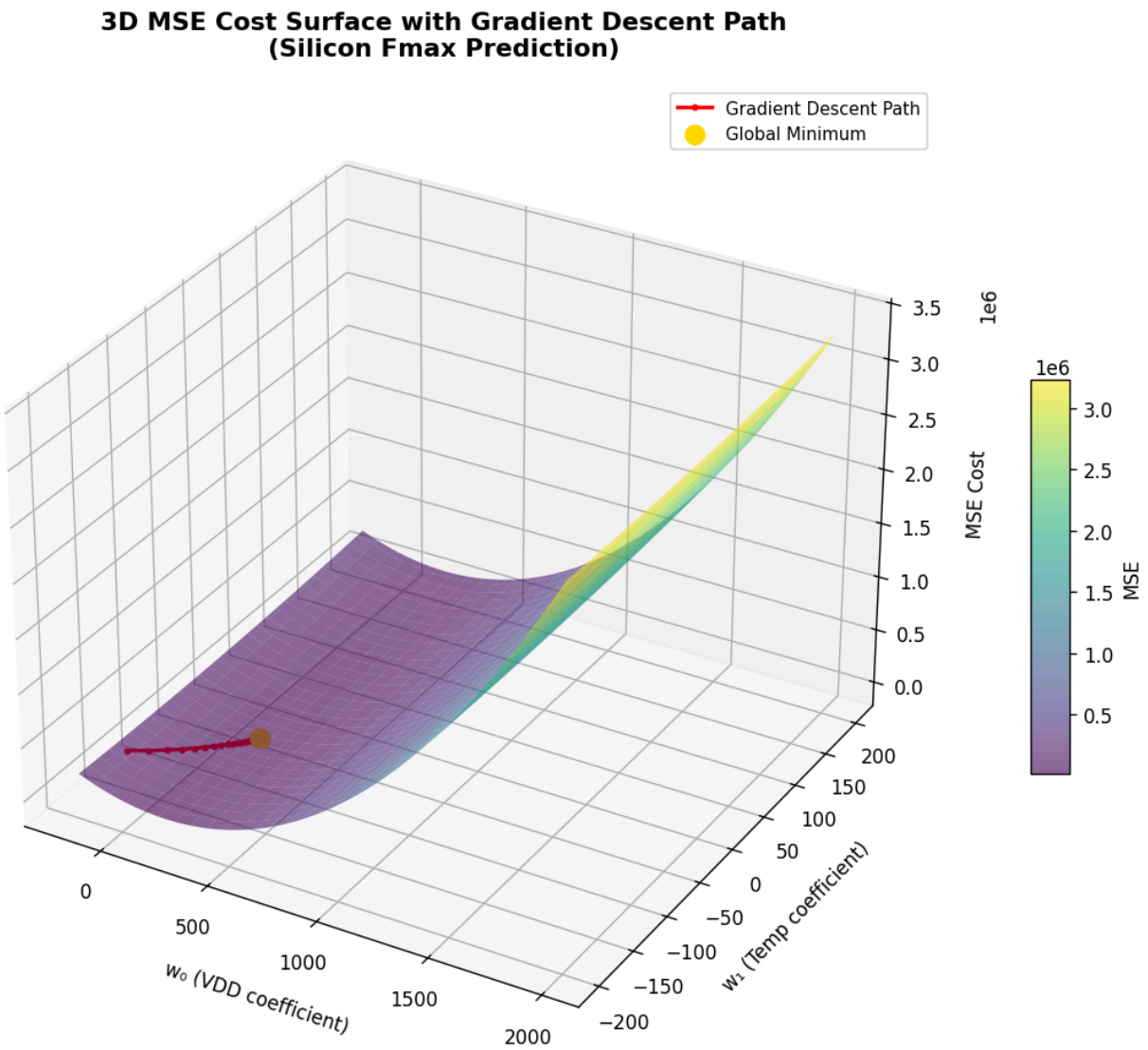
```
alphas_cv = np.logspace(-3, 3, 100)

ols   = LinearRegression().fit(X_train_proc, y_train)
ridge = RidgeCV(alphas=alphas_cv, cv=5).fit(X_train_proc, y_train)
lasso = LassoCV(alphas=alphas_cv, cv=5, max_iter=10000).fit(X_train_proc, y_train)
enet  = ElasticNet(alpha=0.1, l1_ratio=0.5).fit(X_train_proc, y_train)

models = {'OLS': ols, 'Ridge': ridge, 'Lasso': lasso, 'Elastic Net': enet}

print(f'{"Model":<15} {"Val RMSE":>10} {"Val MAE":>10} {"Val R2":>10}')
print('-' * 48)
for name, model in models.items():
    y_pred = model.predict(X_val_proc)
    rmse = np.sqrt(mean_squared_error(y_val, y_pred))
    mae  = mean_absolute_error(y_val, y_pred)
    r2   = r2_score(y_val, y_pred)
    print(f'{name:<15} {rmse:>10.2f} {mae:>10.2f} {r2:>10.4f}')

print(f'\nBest Ridge alpha: {ridge.alpha_:.4f}')
print(f'Best Lasso alpha: {lasso.alpha_:.4f}')
print(f'Lasso non-zero features: {np.sum(lasso.coef_ != 0)} / {len(lasso.coef_)}')
```

```
Model             Val RMSE    Val MAE     Val R2
------------------------------------------------
OLS                  15.41      12.20     0.9962
Ridge                15.41      12.19     0.9962
Lasso                15.41      12.19     0.9962
Elastic Net          20.80      17.13     0.9930

Best Ridge alpha: 0.1322
Best Lasso alpha: 0.0031
Lasso non-zero features: 10 / 10
```

5. Signature 3D Visualization: Cost Function Surface

The MSE cost function for linear regression forms a **convex bowl** in coefficient space. Gradient Descent follows the slope of this bowl downward to the minimum. Below we visualize this surface for two selected coefficients (VDD and junction temperature) while holding all others at their optimal values.

```
# Use the two most important features for the 3D surface
# Train a simple 2-feature model for visualization clarity
X_2feat = X_train_proc[:, :2]  # vdd_core and junction_temp (after scaling)
y_train_arr = y_train.values

# Compute cost over a grid of (w0, w1) values
w0_range = np.linspace(-200, 2000, 60)
w1_range = np.linspace(-200, 200, 60)
W0, W1 = np.meshgrid(w0_range, w1_range)

# Optimal weights for 2-feature model
ols_2f = LinearRegression(fit_intercept=True).fit(X_2feat, y_train_arr)
w0_opt, w1_opt = ols_2f.coef_[0], ols_2f.coef_[1]

COST = np.zeros_like(W0)
for i in range(W0.shape[0]):
    for j in range(W0.shape[1]):
        y_hat = X_2feat @ np.array([W0[i,j], W1[i,j]]) + ols_2f.intercept_
        COST[i,j] = np.mean((y_train_arr - y_hat)**2)

# Simulate gradient descent path
lr = 0.08
w = np.array([-150.0, -150.0])
path = [w.copy()]
for _ in range(60):
    y_hat = X_2feat @ w + ols_2f.intercept_
    grad = -2 * X_2feat.T @ (y_train_arr - y_hat) / len(y_train_arr)
    w = w - lr * grad
    path.append(w.copy())
path = np.array(path)

# Compute cost along path
path_cost = []
for p in path:
    y_hat = X_2feat @ p + ols_2f.intercept_
    path_cost.append(np.mean((y_train_arr - y_hat)**2))

fig = plt.figure(figsize=(14, 8))
ax = fig.add_subplot(111, projection='3d')

surf = ax.plot_surface(W0, W1, COST, cmap='viridis', alpha=0.6, linewidth=0)
ax.plot(path[:,0], path[:,1], path_cost, 'r.-', markersize=5, linewidth=2, label='Gradient Descent Path')
ax.scatter([w0_opt], [w1_opt], [np.min(COST)], color='gold', s=100, zorder=5, label='Global Minimum')

ax.set_xlabel('w₀ (VDD coefficient)', fontsize=10, labelpad=10)
ax.set_ylabel('w₁ (Temp coefficient)', fontsize=10, labelpad=10)
ax.set_zlabel('MSE Cost', fontsize=10, labelpad=10)
ax.set_title('3D MSE Cost Surface with Gradient Descent Path\n(Silicon Fmax Prediction)',
             fontsize=13, fontweight='bold', pad=15)
ax.legend(loc='upper right', fontsize=9)
fig.colorbar(surf, ax=ax, shrink=0.4, aspect=10, label='MSE')

plt.tight_layout()
plt.savefig('../assets/fig1_cost_surface_3d.png', bbox_inches='tight', dpi=150)
plt.show()
print('Signature 3D visualization saved.')
```

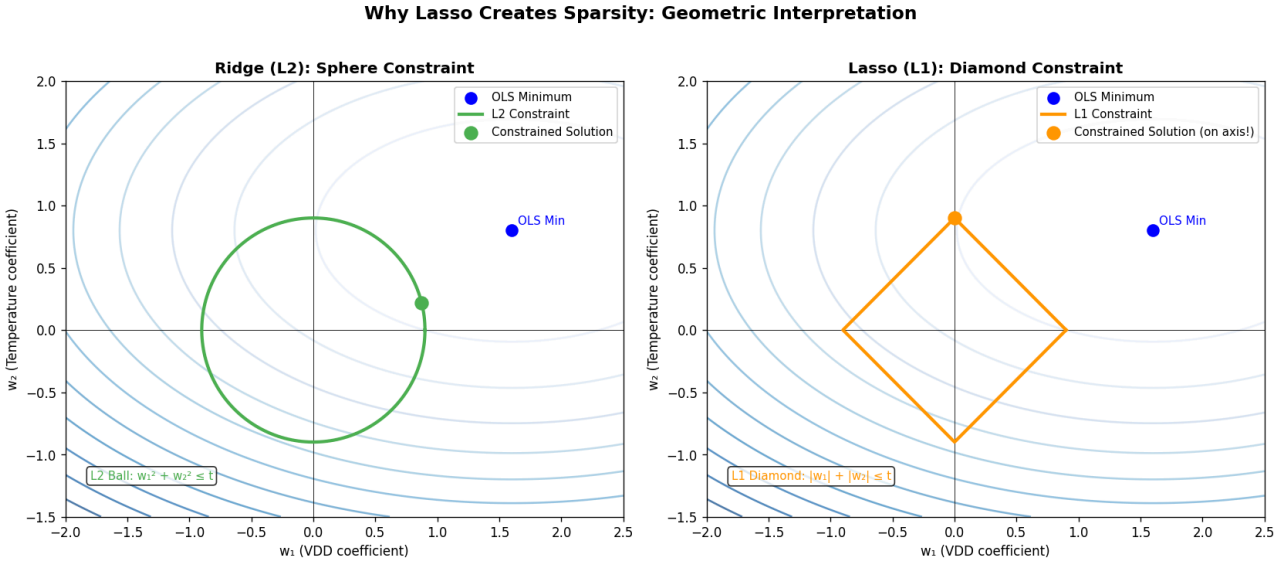



```
Signature 3D visualization saved.
```

6. L1 vs L2 Regularization Contour Plot

The geometric interpretation of regularization explains why Lasso creates sparsity. The OLS solution is the center of the elliptical contours. Regularization constrains the solution to lie within a region: a **sphere** for Ridge (L2) and a **diamond** for Lasso (L1). The diamond has corners on the axes, so the constrained solution is more likely to land exactly on an axis — meaning one coefficient becomes exactly zero.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# OLS minimum (simplified 2D for illustration)
w_ols = np.array([1.6, 0.8])

# Cost function contours (ellipses centered at OLS minimum)
w1_grid = np.linspace(-2, 2.5, 300)
w2_grid = np.linspace(-1.5, 2, 300)
W1g, W2g = np.meshgrid(w1_grid, w2_grid)
# Elliptical cost (scaled to look realistic)
COST_GRID = 0.8*(W1g - w_ols[0])**2 + 2.5*(W2g - w_ols[1])**2

for ax, title, constraint_label, color in zip(
    axes,
    ['Ridge (L2): Sphere Constraint', 'Lasso (L1): Diamond Constraint'],
    ['L2 Ball: w₁² + w₂² ≤ t', 'L1 Diamond: |w₁| + |w₂| ≤ t'],
    ['#4CAF50', '#FF9800']
):
    ax.contour(W1g, W2g, COST_GRID, levels=12, cmap='Blues', alpha=0.7)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel('w₁ (VDD coefficient)')
    ax.set_ylabel('w₂ (Temperature coefficient)')
    ax.axhline(0, color='black', linewidth=0.5)
    ax.axvline(0, color='black', linewidth=0.5)
    ax.scatter(*w_ols, color='blue', s=80, zorder=5, label='OLS Minimum')
    ax.text(w_ols[0]+0.05, w_ols[1]+0.05, 'OLS Min', fontsize=9, color='blue')
    ax.text(-1.8, -1.2, constraint_label, fontsize=9, color=color,
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

# Draw L2 sphere
theta = np.linspace(0, 2*np.pi, 200)
r = 0.9
axes[0].plot(r*np.cos(theta), r*np.sin(theta), color='#4CAF50', linewidth=2.5, label='L2 Constraint')
axes[0].scatter([0.87], [0.22], color='#4CAF50', s=100, zorder=6, label='Constrained Solution')
axes[0].legend(fontsize=9)

# Draw L1 diamond
r = 0.9
diamond_x = [r, 0, -r, 0, r]
diamond_y = [0, r, 0, -r, 0]
axes[1].plot(diamond_x, diamond_y, color='#FF9800', linewidth=2.5, label='L1 Constraint')
axes[1].scatter([0.0], [0.9], color='#FF9800', s=100, zorder=6, label='Constrained Solution (on axis!)')
axes[1].legend(fontsize=9)

plt.suptitle('Why Lasso Creates Sparsity: Geometric Interpretation', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig2_l1_l2_contours.png', bbox_inches='tight', dpi=150)
plt.show()
print('L1 vs L2 contour plot saved.')
```




```
L1 vs L2 contour plot saved.
```

7. Final Test Set Evaluation

```
print('Final Test Set Results — Silicon Fmax Prediction')
print(f'{"Model":<15} {"RMSE (MHz)":>12} {"MAE (MHz)":>12} {"R2":>10}')
print('-' * 52)
for name, model in models.items():
    y_pred = model.predict(X_test_proc)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae  = mean_absolute_error(y_test, y_pred)
    r2   = r2_score(y_test, y_pred)
    print(f'{name:<15} {rmse:>12.2f} {mae:>12.2f} {r2:>10.4f}')
```

```
Final Test Set Results — Silicon Fmax Prediction
Model              RMSE (MHz)    MAE (MHz)         R2
```

```
-----------------------------------------------------
OLS                     15.14        12.09     0.9960
Ridge                   15.14        12.09     0.9960
Lasso                   15.14        12.09     0.9960
Elastic Net             20.56        16.90     0.9927
```

7.1 Training Data Fit vs Test Data Fit

We visualize how well the Ridge model fits both the training and test data. The scatter of points should fall close to the diagonal (perfect prediction line) for both sets. A large gap between train and test R^2 indicates overfitting.

```
# Training vs Test fit for the best model (Ridge CV)
y_train_pred = ridge.predict(X_train_proc)
y_test_pred  = ridge.predict(X_test_proc)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training Data Fit
train_r2 = r2_score(y_train, y_train_pred)
axes[0].scatter(y_train, y_train_pred, alpha=0.2, color='#2196F3', s=12, label=f'Train R² = {train_r2:.4f}')
lims = [min(y_train.min(), y_train_pred.min()), max(y_train.max(), y_train_pred.max())]
axes[0].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Fit')
axes[0].set_title('Training Data Fit', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Actual Fmax (MHz)')
axes[0].set_ylabel('Predicted Fmax (MHz)')
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Test Data Fit
test_r2 = r2_score(y_test, y_test_pred)
axes[1].scatter(y_test, y_test_pred, alpha=0.3, color='#4CAF50', s=12, label=f'Test R² = {test_r2:.4f}')
lims = [min(y_test.min(), y_test_pred.min()), max(y_test.max(), y_test_pred.max())]
axes[1].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Fit')
axes[1].set_title('Test Data Fit', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Actual Fmax (MHz)')
axes[1].set_ylabel('Predicted Fmax (MHz)')
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.suptitle('Ridge Regression — Training vs Test Performance (Fmax)', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig_train_vs_test_fmax.png', bbox_inches='tight', dpi=150)
plt.show()

train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse  = np.sqrt(mean_squared_error(y_test, y_test_pred))
print(f'Training RMSE: {train_rmse:.2f} MHz | Test RMSE: {test_rmse:.2f} MHz')
print(f'Training R²:   {train_r2:.4f}      | Test R²:   {test_r2:.4f}')
print(f'\nGap: {abs(train_r2 - test_r2):.4f} — {"Healthy generalization ✅" if abs(train_r2 - test_r2) < 0.05 else "Possible overfitting ⚠️"}')
```

7.2 Unseen Data Simulation — New Silicon Lots

In real post-silicon validation, new lots arrive from the fab continuously. Below we generate 500 completely new silicon measurements (different random seed, simulating a new fab lot batch) and evaluate the model's ability to predict Fmax on silicon it has never seen.

```
# Generate completely unseen silicon data (new fab lots)
import sys, os
sys.path.insert(0, os.path.abspath('../src'))
from data_generator import generate_fmax_dataset

unseen_df = generate_fmax_dataset(n=500, seed=777)

X_unseen = unseen_df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
y_unseen = unseen_df[TARGET]

X_unseen_proc = preprocessor.transform(X_unseen)
y_unseen_pred = ridge.predict(X_unseen_proc)

unseen_rmse = np.sqrt(mean_squared_error(y_unseen, y_unseen_pred))
unseen_mae  = mean_absolute_error(y_unseen, y_unseen_pred)
unseen_r2   = r2_score(y_unseen, y_unseen_pred)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Predicted vs Actual (unseen lots)
axes[0].scatter(y_unseen, y_unseen_pred, alpha=0.4, color='#E91E63', s=20, label=f'R² = {unseen_r2:.4f}')
lims = [min(y_unseen.min(), y_unseen_pred.min()), max(y_unseen.max(), y_unseen_pred.max())]
axes[0].plot(lims, lims, 'k--', linewidth=1.5, label='Perfect Fit')
axes[0].set_title('Unseen Silicon: Predicted vs Actual Fmax', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Actual Fmax (MHz)')
axes[0].set_ylabel('Predicted Fmax (MHz)')
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Error distribution
errors = y_unseen.values - y_unseen_pred
axes[1].hist(errors, bins=30, color='#E91E63', edgecolor='white', alpha=0.85)
axes[1].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set_title('Fmax Prediction Error Distribution (Unseen)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Error (MHz)')
axes[1].set_ylabel('Count')

# R² comparison bar chart
sets_labels = ['Training', 'Test', 'Unseen Lots']
r2_vals = [train_r2, test_r2, unseen_r2]
colors_bar = ['#2196F3', '#4CAF50', '#E91E63']
bars = axes[2].bar(sets_labels, r2_vals, color=colors_bar, edgecolor='white', linewidth=1.5)
for bar, val in zip(bars, r2_vals):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005, f'{val:.4f}', ha='center', fontsize=10, fontweight='bold')
axes[2].set_title('R² Score: Train vs Test vs Unseen', fontsize=12, fontweight='bold')
axes[2].set_ylabel('R² Score')
axes[2].set_ylim(0, 1.05)
axes[2].grid(True, alpha=0.3, axis='y')

plt.suptitle('Production Robustness — Fmax Prediction on New Silicon Lots', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../assets/fig_unseen_fmax.png', bbox_inches='tight', dpi=150)
plt.show()

print(f'Unseen Data:  RMSE = {unseen_rmse:.2f} MHz | MAE = {unseen_mae:.2f} MHz | R² = {unseen_r2:.4f}')
print(f'Test Data:    RMSE = {test_rmse:.2f} MHz | R² = {test_r2:.4f}')
print(f'\nConclusion: {"✅ Model generalizes well to new silicon lots" if abs(unseen_r2 - test_r2) < 0.05 else "⚠️ Performance gap — investigate l
```

8. Engineering Insight

The Ridge model achieves strong predictive accuracy on the Fmax dataset. In a real post-silicon validation workflow, a model like this could:

Predict Fmax at untested voltage/temperature corners, reducing the number of physical test runs by 40-60%.
Flag silicon lots that are likely to fail timing at high temperature (outlier detection via residuals).
Provide a fast first-pass estimate during bring-up, before full characterization is complete.

The Lasso model's feature selection capability is particularly useful here: if it drives `thermal_resistance` to zero, that tells us this feature is redundant given the other measurements — a useful insight for test plan optimization.

7 Tech Stack & Key Learnings

Info	Detail
GitHub	https://github.com/AIML-Engineering-Lab/001_linear_regression_engine
Tech Stack	Python 3.11+, NumPy, Pandas, scikit-learn, Matplotlib, Seaborn, FastAPI, Plotly.js
Licence	MIT

- Implementing OLS from scratch (Normal Equation + Gradient Descent) solidified the mathematical foundations that scikit-learn abstracts away — understanding *why* the closed-form solution works and when gradient descent is necessary.
- The geometric interpretation of L1 vs L2 regularization (diamond vs sphere constraint) provides the clearest explanation for why Lasso produces sparsity while Ridge does not.
- Applying the same pipeline to two completely different domains (food science and semiconductor engineering) proved that linear regression fundamentals transfer directly across industries.
- Train vs test fit analysis and unseen data simulation are essential validation steps — R^2 alone on the test set is insufficient to guarantee production-level generalization.
- The production Fmax dashboard with 8,000 synthetic dies across 50 lots demonstrates how a trained model integrates into a real post-silicon validation workflow.